

0.111. Grand — l'autre nom d'un Perso de 32 × 32

`#include <Grand>` : **Grand NOM** = { trente-deux rangées }; l'autre nom d'un Perso de 32 × 32. Un Perso de trente-deux rangées est déjà un 32 × 32 : **Grand** ne sert qu'à le dire dans le nom du type.

1. Ce qu'il faut comprendre

Un personnage de 32 × 32 est un Perso : `Perso BOSS = { ... }` avec trente-deux rangées de trente-deux signes, et `#include <Perso>`. **Grand** en est l'autre nom, pour qui veut que le type dise la taille. Les deux donnent exactement la même chose.

Ce que c'est : un **Grand** est un dessin de **32 × 32 pixels** : trente-deux rangées de trente-deux signes. C'est quatre fois un `Perso` : le compilateur le coupe en **quatre quarts de 16 × 16**, rangés l'un après l'autre (haut-gauche, haut-droite, bas-gauche, bas-droite), soit **seize tuiles**.

La ligne à écrire : `#include <Grand>`. Un Grand écrit ne s'affiche pas tout seul : c'est un dessin rangé, prêt à servir. `sprite32()` le montre à l'écran (chapitre 6, avec les personnages).

Ce qu'il coûte : **256 octets** de dessin (seize tuiles de seize octets) — gravés quand on s'en sert.

Dans l'atelier : «  Les tuiles », bouton « **+ perso 32 × 32** » : on choisit la taille AVANT de dessiner, puis on peint dans une grille de 32 × 32 ; l'atelier écrit un `Perso`.

Essaie : dans l'onglet `perso_GEANT.cpp`, écris `Perso GEANT` au lieu de `Grand GEANT` (et, dans la liste, `#include <Perso>`) : c'est exactement le même personnage.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **820 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	29 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	528 o	← CopierTuiles — 17 lettres sur 43, et 16 tuiles dessinées
"UN GRAND ECRIT"	14 o	← texte() — un texte gravé
"SPRITE32 LE MON..."	18 o	← texte() — un texte gravé

Lettres gravées : ACDEGILMNOPRSTU23

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,

LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte() × 2 22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1 3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <Grand> : un grand personnage de 32 × 32 ----  
// GEANT est dessiné dans perso_GEANT.cpp, en Grand ; ici, on le verse.
```

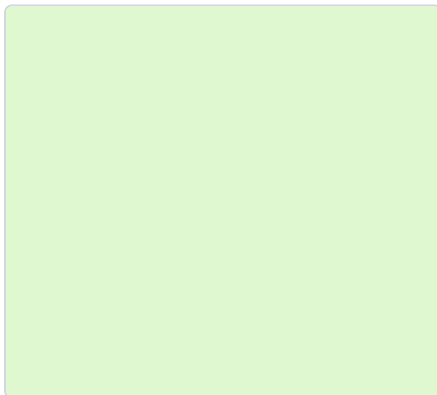
```
#include <texte>  
#include "personnages.cpp"    // la liste : GEANT
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

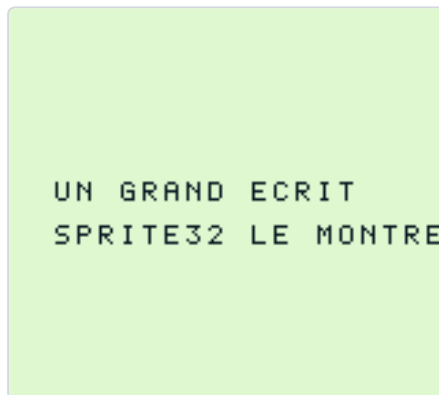
```
int main() {  
    texte(2, 8, "UN GRAND ECRIT");  
    texte(2, 10, "SPRITE32 LE MONTRE");  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

UN GRAND ECRIT : le dessin compile ; l'étape suivante le montre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le dessin de 32 × 32 compile

